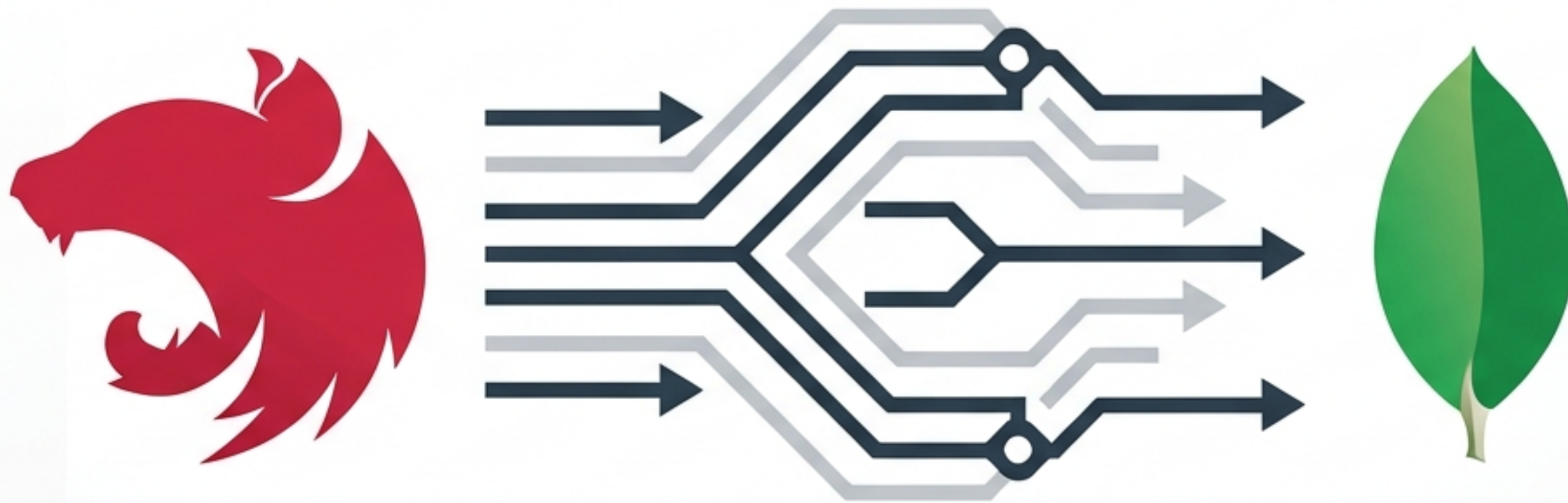




Configuración de MongoDB en NestJS

Guía Práctica de Implementación: Modelado, Inyección y Exposición REST API.

El Objetivo: ¿Qué vamos a construir?

Implementar una feature mínima para registrar usuarios. Conectaremos NestJS con MongoDB para recibir un DTO, procesarlo y guardar un documento con su identificador único.

Endpoint: POST /users

```
{
  "email": "test@mail.com",
  "name": "Sebastian"
}
```

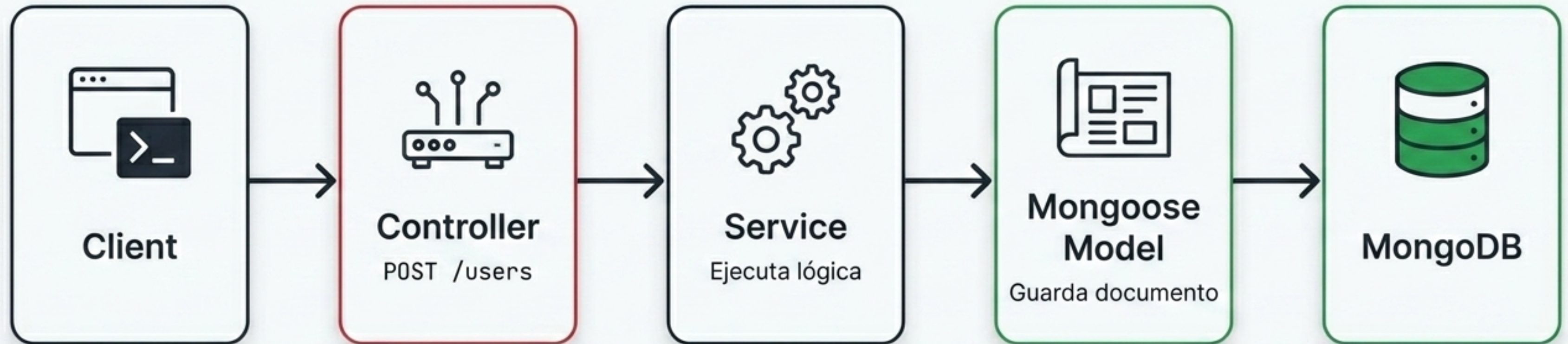


Respuesta de MongoDB

```
{
  "_id": "6637f8...",
  "email": "test@mail.com",
  "name": "Sebastian",
  "createdAt": "2026-05-04T23:00:00.000Z",
  "updatedAt": "2026-05-04T23:00:00.000Z",
  "__v": 0
}
```


Arquitectura y Flujo de Datos

El ciclo de vida de la petición. En esta configuración inicial, el Service interactúa directamente con el modelo de Mongoose.



Evolución de la Arquitectura

Arquitectura Actual (Clase de hoy)	Arquitectura Completa (Futuro)
Flujo directo: Controller → Service → Mongo Ideal para prototipos rápidos y entender la inyección básica de dependencias.	Flujo escalable: Controller → Service → Repository → DAO → Mongo Patrones avanzados (como bcrypt, JWT, y DAO) para aislar la lógica de negocio de la base de datos.

Preparando el Entorno: Dependencias

```
npm install @nestjs/mongoose mongoose
```

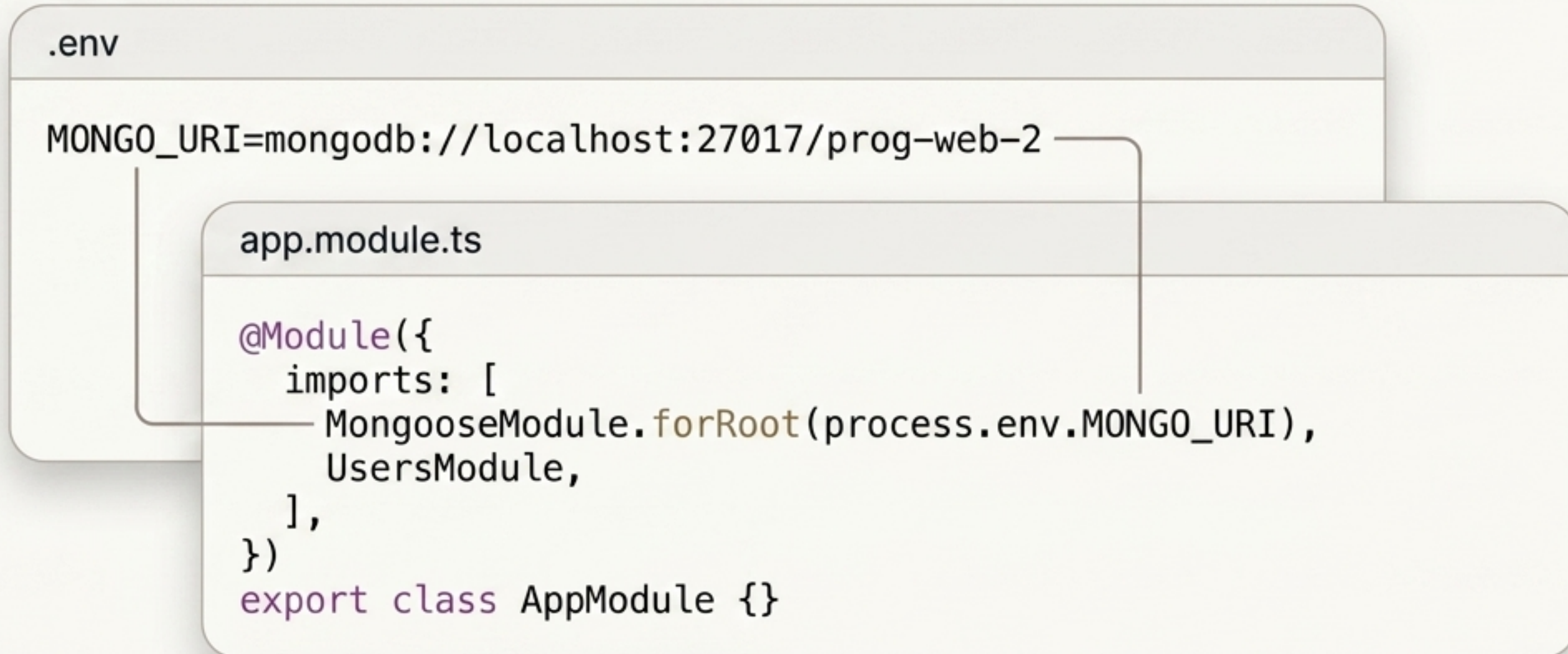
Integración

El módulo oficial que conecta el sistema de Inyección de Dependencias de NestJS con Mongoose.

El ODM (Object Data Modeling)

La librería core que permite trabajar con MongoDB usando clases, schemas y modelos tipados.

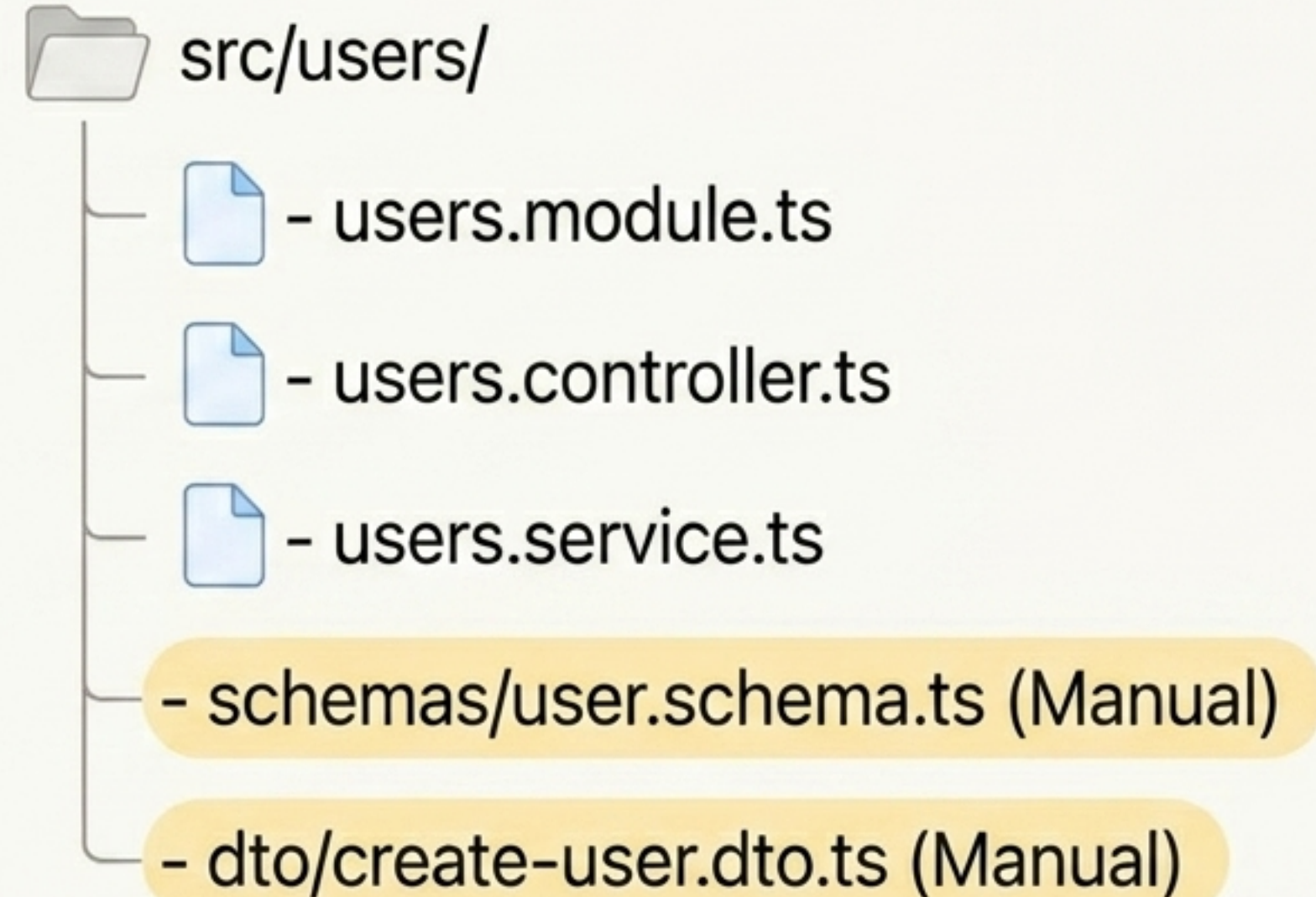
Configuración Segura: Variables de Entorno



¡Nunca **hardcodees** credenciales! En proyectos reales, usa variables de entorno para mantener segura la conexión al servidor Mongo local (**localhost:27017**) y tu base de datos (**prog-web-2**).

Generando la Estructura Base (CLI)

```
nest g module users  
nest g controller users  
nest g service users
```



Modelado de Datos: El Schema (Parte 1)

user.schema.ts

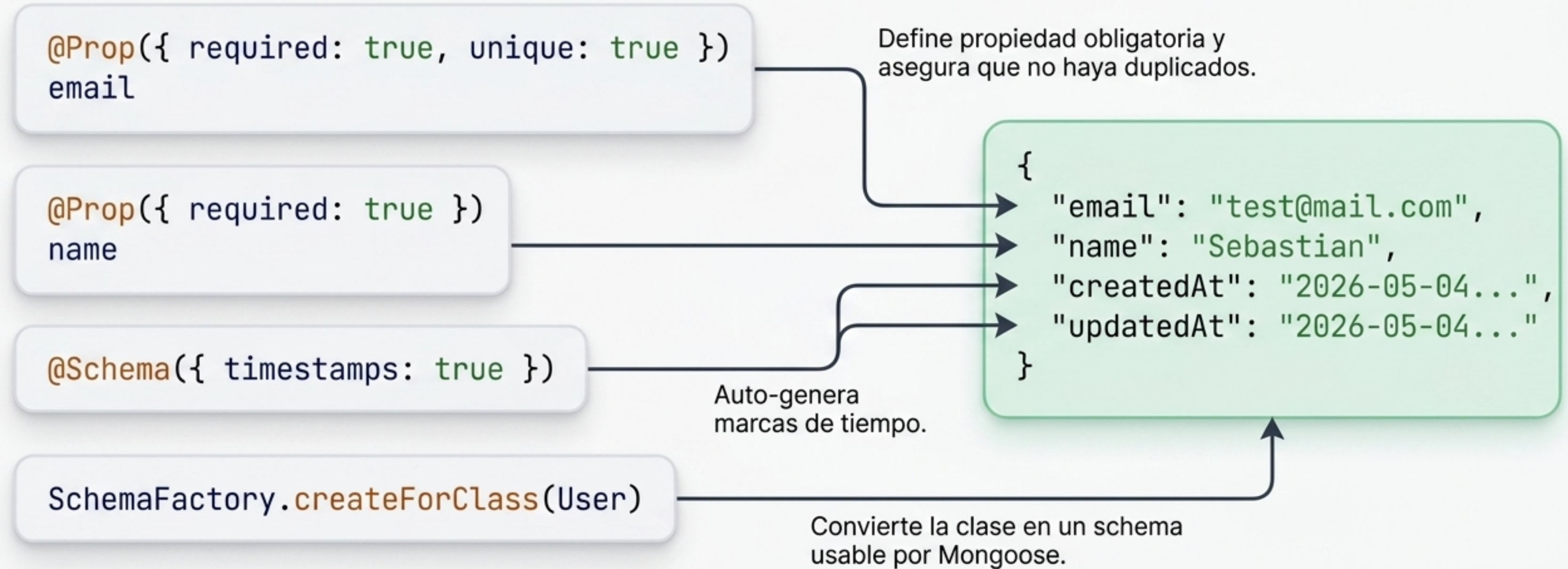
```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { HydratedDocument } from 'mongoose';

export type UserDocument = HydratedDocument<User>;

@Schema({ timestamps: true })
export class User {
  @Prop({ required: true, unique: true })
  email: string;
  @Prop({ required: true })
  name: string;
}

export const UserSchema = SchemaFactory.createForClass(User);
```


Modelado de Datos: Traducción a Documentos



Registrando el Modelo en el Módulo



users.module.ts

```
@Module({
  imports: [
    mongooseModule.forFeature([
      { name: User.name, schema: UserSchema },
    ]),
  ],
  controllers: [UsersController],
  providers: [UsersService],
})
export class UsersModule {}
```

Este paso es crucial. Al registrar UserSchema con forFeature, le decimos a NestJS que el modelo existe y puede ser inyectado en nuestro UsersService.

El Contrato de Datos: DTO

create-user.dto.ts

```
export class CreateUserDto {  
  email: string;  
  name: string;  
}
```

Pro-Tip

Mejora Futura:

Para mantener el foco en Mongo, este DTO es simple. En una aplicación real, se deben usar decoradores de class-validator como **@IsEmail()** y **@IsString()** para validar la data antes de que llegue al controlador.

Lógica de Negocio: El Service

```
users.service.ts

@Injectable()
export class UsersService {
  constructor(
    @InjectModel(User.name) private readonly userModel: Model<UserDocument>,
  ) {}

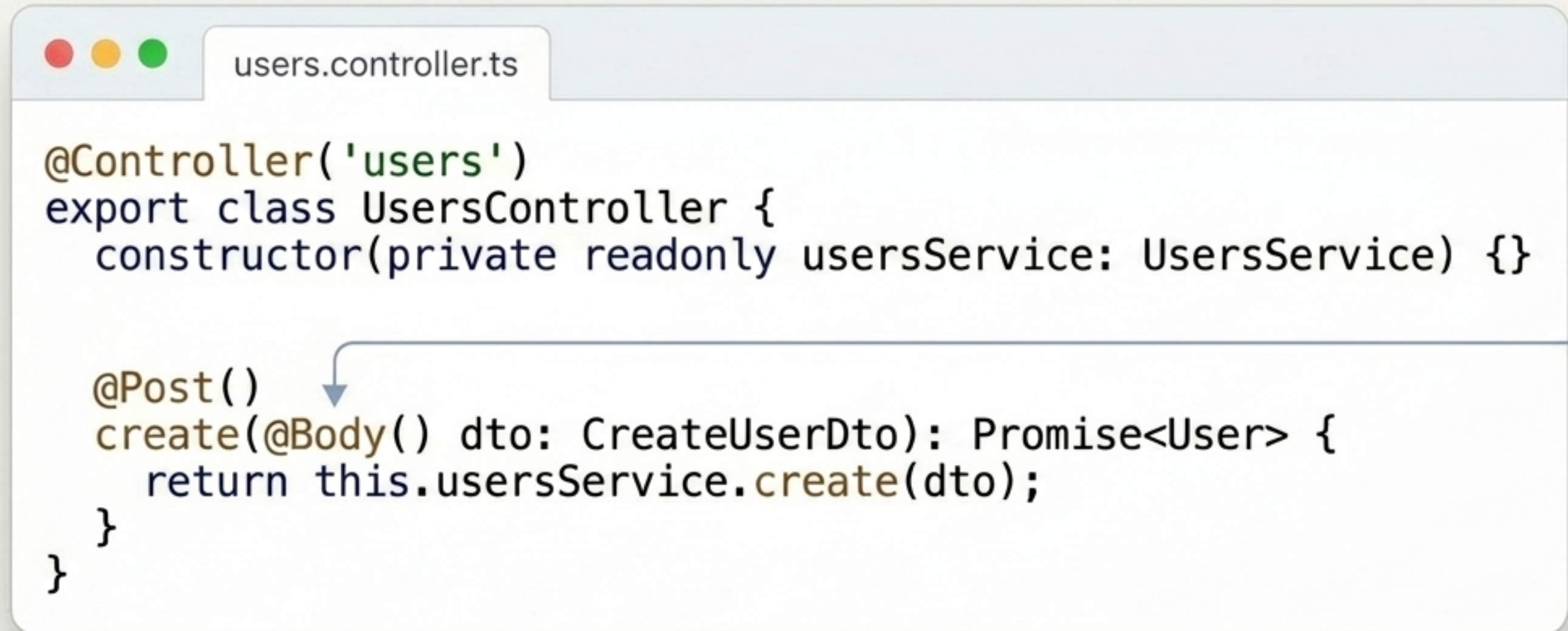
  async create(dto: CreateUserDto): Promise<User> {
    const existingUser = await this.userModel.findOne({ email: dto.email }).exec(); 1
    if (existingUser) throw new ConflictException('Email already exists');

    return this.userModel.create({ email: dto.email, name: dto.name }); 2
  }
}
```

Step 1: `findOne` verifica duplicados.

Step 2: `create` guarda el nuevo usuario.

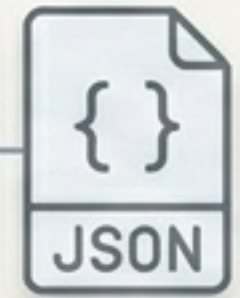
Exposición HTTP: El Controller



```
users.controller.ts

@Controller('users')
export class UsersController {
    constructor(private readonly userService: UsersService) {}

    @Post()
    create(@Body() dto: CreateUserDto): Promise<User> {
        return this.userService.create(dto);
    }
}
```



El controlador expone el endpoint POST /users, inyecta el UsersService, e intercepta el cuerpo de la petición (@Body) mapeándolo a nuestro CreateUserDto.

La Prueba Final

Server Start

```
> npm run start:dev  
[NestApplication] Nest application successfully started
```

Request

```
> curl -X POST http://localhost:3000/users -H "Content-Type: application/json" -d '{  
  "email": "test@mail.com", "name": "Sebastian" }'
```

Response

```
{  
  "_id": "6637f8...",  
  "email": "test@mail.com",  
  "name": "Sebastian",  
  "createdAt": "2026-05-04T23:00:00.000Z",  
  "updatedAt": "2026-05-04T23:00:00.000Z",  
  "__v": 0  
}
```


Síntesis del Flujo



1

Conexión Segura: MongoDB integrado mediante MongooseModule y .env.

2

Modelado Tipado: Schemas de Mongoose fuertemente acoplados a clases de TypeScript.

3

Inyección Limpia: El modelo es registrado (forFeature) e inyectado globalmente en el ecosistema NestJS para exponer una API REST robusta.